Taxonomizing Adversarial Prompts: An Unsupervised Approach to LLM Attack Coverage Analysis

Rishitha Pamu

July 28, 2026

1 Introduction

Large language models are probed continuously for weaknesses using adversarial prompts — inputs deliberately constructed to bypass safety training/guardrails and elicit disallowed content. In the past three years, several independent research groups have released benchmark datasets of such prompts: JailbreakBench (1), AdvBench (2), HarmBench (3), Do-Not-Answer (4), and community-collected corpora such as In-The-Wild (5). The datasets were created for different purposes— such as evaluating a defense, benchmarking an attack method, or auditing model refusal behaviour— and use their own labeling schemes, where applicable.

A *jailbreak*, in this context, refers specifically to a prompt or sequence of prompts engineered to cause a model to bypass its own safety training and produce a response it would otherwise refuse– as distinct from a prompt that is simply harmful in content but makes no attempt to circumvent a refusal. The open-source fuzzing tool FuzzyAI (**?**), for instance, catalogs and automates close to twenty distinct jailbreak techniques, ranging from persona-adoption attacks such as DAN ("Do Anything Now") and history/academic framing, to structural attacks such as ASCII-art obfuscation (ArtPrompt), many-shot context flooding, and genetic-algorithm-generated adversarial suffixes, to multi-turn conversational attacks such as Crescendo and ActorAttack that escalate gradually across several exchanges rather than relying on a single adversarial prompt. It is also a useful point of comparison for this project's scope: the five datasets ingested here are static text-prompt collections, and do not capture the multi-turn, iterative, or tool-assisted jailbreak techniques that fuzzing frameworks like FuzzyAI are built to automate — a scope limitation discussed further in Section 5.

This fragmentation creates a practical problem for AI security research: there is no unified view of *what kinds of attacks these datasets actually contain*, how much they overlap, or where the collective blind spots are. A defense system evaluated only against one benchmark may appear robust, while remaining naive to attack patterns well represented in another. Conversely, a researcher designing a new benchmark has no principled way to identify which attack technique–objective combinations are already saturated versus genuinely under-explored.

This project addresses that gap directly. We built a pipeline that:

1. normalizes five heterogeneous datasets into a single schema without discarding source-specific information,
2. embeds and clusters the combined corpus by semantic and behavioral similarity,
3. removes the duplicates first and then assigns each cluster to two orthogonal taxonomies — *how* the attack is constructed and *what* harm it targets
4. computes a coverage matrix over these taxonomies to surface structural gaps in the combined dataset.

The result is not a new attack method or a new defense, but an empirical map of the existing adversarial-prompt landscape: what is well represented, what is not, and where the apparent diversity in these datasets is more illusory (duplicated, or dominated by a small number of meta-techniques) than it first appears. Throughout Section 3, we deliberately foreground the alternatives considered and rejected at each stage, since a pipeline design is best understood not just by what it does, but by what it was chosen over.

2 Related Work

Adversarial prompt benchmarks. AdvBench (2) was among the earliest systematic collections of harmful-behavior prompts, released without a category schema — it treats attack success as something the evaluator must judge, not something the dataset itself labels. HarmBench (3) improved on this by introducing a two-dimensional label schema of its own (semantic category and functional attack type), explicitly built for benchmarking attack methods against defenses. JailbreakBench (1) is notable for including *benign* prompts alongside harmful

ones, enabling false-positive (over-refusal) measurement, and for being partially derived from both AdvBench and HarmBench prompts — a provenance detail that becomes directly relevant to our duplicate-detection findings in Section xx. Do-Not-Answer (4) is unusual in labeling not just the prompt but the *expected model response type*, making it useful for response-quality evaluation rather than refusal-rate counting alone. In-The-Wild (5) departs from the academic datasets entirely: its prompts are scraped from Reddit, Discord, and other community sources, and are labeled only with platform metadata and jailbreak-confirmation status, with no behavioral or harm categorization.

Datasets considered but not ingested. Several other public resources are relevant to this problem space but fall outside the scope of the current corpus, either due to scale, focus, or timing relative to the internship. WildJailbreak (6) is substantially larger than any dataset used here — over 50K harmful queries paired with more than 13K crowd-sourced jailbreak tactics, generated via the WildTeaming framework by recombining harmful queries with real-world jailbreak strategies. Its "vanilla benign" subset, over 50K prompts designed to superficially resemble unsafe queries without being harmful, would be directly useful for extending our behavior taxonomy's coverage of over-refusal edge cases, a dimension the current five-dataset corpus does not test explicitly. XSTest (7) takes a narrower but complementary angle: 450 prompts split between 250 safe prompts (across ten categories designed to *look* unsafe) and 200 genuinely unsafe prompts, purpose-built to measure exaggerated safety refusals rather than jailbreak success — a distinct research question from the technique/objective coverage this project focuses on, but a natural extension if the registry were adapted to score defenses rather than characterize attack prompts. ToxicChat (8) differs in kind rather than degree: its $2{,}853$ prompts are drawn from real user interactions with a deployed chatbot (Vicuna) rather than constructed or curated by researchers, and are annotated for subtle or indirect toxic intent rather than explicit harmful requests — a source of naturalistic, in-production adversarial behavior that neither our academic datasets nor In-The-Wild's community-forum prompts fully capture. RealToxicityPrompts (9) predates the instruction-following jailbreak paradigm entirely, evaluating toxic degeneration in open-ended text continuation rather than adversarial instruction-prompts, and was excluded on those grounds — its attack surface (completion toxicity) does not map onto the primitive/behavior registry designed around instruction-based attacks.

These four datasets were not ingested in the current iteration primarily due to scope: WildJailbreak alone is more than an order of magnitude larger than the entire combined corpus used here, and incorporating it would have shifted the project from characterizing an existing, tractable corpus toward a substantially larger-scale data engineering effort better suited to a longer engagement. They are flagged here, and in Section 6, as the most promising next additions for extending this project's coverage analysis.

Clustering-based analysis of prompt corpora. Unsupervised clustering (via dimensionality reduction and density-based methods such as HDBSCAN (10)) is a standard tool for discovering latent structure in text corpora without requiring labeled data upfront. Its use here is motivated by the absence of a shared label schema across our five source datasets — since no common ground-truth taxonomy exists, clustering by semantic similarity is the only way to group prompts by *behavioral* similarity rather than by which dataset happened to contain them.

This work's contribution. Where prior work labels prompts *within* a single dataset, we contribute a method for aligning behaviorally similar prompts *across* datasets, and a two-axis taxonomy (primitive $\times$ behavior) that makes coverage gaps directly visible as a matrix, rather than as prose observations about any one dataset in isolation.

3 Methods

Each stage of this pipeline involved a genuine choice between multiple viable approaches. Rather than presenting only the final design, this section documents the alternatives considered at each stage and the reasoning that ruled them out, since the rejected options are often as informative as the chosen one.

3.1 Decision: Which Dataset to Ingest First

Alternatives considered. With five candidate datasets of very different size, license, and label richness (Table 1), the pipeline could have been built against any one of them first, and the choice determines which assumptions get baked into the normalized schema.

Options rejected. Building first against In-The-Wild was rejected despite its real-world provenance, because its prompts are inconsistently labeled and it was unclear at the outset whether a normalized schema built around such minimal metadata would generalize to the richer academic datasets. Building first against HarmBench or Do-Not-Answer was rejected because their labeling schemes, while rich, are dataset-specific — a schema designed around either would risk over-fitting the normalized `AttackRecord` format to one dataset's idiosyncratic categories.

Decision taken. JailbreakBench was ingested first. It is small enough to iterate on quickly and critically, it is itself partially derived from both AdvBench and HarmBench, meaning a schema built to accommodate it is more likely

Table 1: Datasets considered for initial pipeline design.

Dataset	Size	Labels	Real-world	Benign prompts
AdvBench	1,096	None	No	No
HarmBench	400+	Rich	No	No
JailbreakBench	200	Medium	No	Yes
Do-Not-Answer	939	Rich	No	No
In-The-Wild	6,387	Minimal	Yes	No

to already accommodate its sources. This decision proved consequential: it is precisely this derivation relationship that the cross-source deduplication analysis later empirically confirms as data leakage (Section 4.3). In-The-Wild was deliberately ingested second, once the schema was stable, specifically to stress-test it against the messiest, least-labeled dataset before considering the schema finalized.

3.2 Decision: Schema Design for Heterogeneous Sources

Alternatives considered. Three options were available for representing five structurally different datasets: (a) keep each dataset in its native format and write dataset-specific downstream code, (b) normalize aggressively into a schema with only the fields common to all five datasets, or (c) normalize into a shared schema with per-field optionality plus a lossless fallback.

Options rejected. Option (a) was rejected because it would require every downstream stage — embedding, clustering, registry assignment — to special-case each dataset, multiplying the surface area for bugs by five. Option (b) was rejected because the intersection of fields common to all five datasets is nearly empty (only `prompt` and `source` survive intersection); this would have discarded HarmBench's semantic/functional categories and JailbreakBench's target-behavior labels entirely, destroying information the registry stage would later need.

Decision taken. Option (c): a single Pydantic-validated `AttackRecord` schema where dataset-specific fields (`target_behavior`, `attack_category`, `severity`) are optional and default to `None` where a source dataset cannot populate them, combined with a required `raw: dict` field that losslessly preserves the complete original record. This means the schema can be extended later — e.g., to make use of a field not currently mapped — by re-deriving from `raw` rather than re-downloading and re-ingesting the source dataset from scratch.

A related sub-decision was how to generate record identifiers. A simple auto-incrementing integer was rejected because it is not stable across re-ingestion runs (the same prompt could get a different ID next time, breaking the embedding cache). A random UUID was rejected for the same reason: it is not deterministic given the same input. The chosen approach — a SHA-256 hash of the `(source, prompt)` pair — is deterministic (idempotent re-ingestion, effective embedding-cache hits) and treats identical prompt text from two different sources as two distinct records, which is precisely the property needed to detect cross-source duplication rather than silently collapsing it.

Finally, ingestion wraps each dataset row in its own try/except block rather than one try/except around the entire dataset loop. The rejected alternative — a single outer try/except — would mean one malformed row (e.g., an AdvBench row with an empty prompt field) aborts ingestion of the other several hundred valid rows in that dataset. Isolating failures per-row, paired with an expected/ingested/dropped audit trail, means data loss is quantified rather than catastrophic: $3{,}464$ of $3{,}560$ expected records were ultimately ingested across all five sources.

3.3 Decision: Embedding Model Selection

Alternatives considered. Larger sentence-transformer models (e.g., `all-mpnet-base-v2`, 768-dimensional) and hosted embedding APIs (e.g., OpenAI's `text-embedding-3`) were both viable options, typically offering marginally higher embedding quality than a compact MiniLM model.

Options rejected. The larger local model was rejected primarily on iteration speed: with a corpus of only a few thousand short prompts, the marginal quality gain of a larger model was judged unlikely to change cluster interpretability, while roughly doubling embedding and UMAP runtime during the parameter-sweep stage where fast iteration mattered most. A hosted embedding API was rejected outright given the sensitivity of the corpus content (adversarial and, in places, harmful prompt text) — sending this corpus to a third-party API was judged an unnecessary and avoidable data-handling risk when a local model performs adequately.

Decision taken. `all-MiniLM-L6-v2` (384-dimensional, fully local) was used for all embeddings, cached by record ID so that re-running the pipeline or adding a new dataset only requires embedding the newly added prompts rather than the full corpus.

3.4 Decision: Deduplication Threshold and Method

Alternatives considered. For similarity search, a brute-force pairwise cosine-similarity comparison was considered as the simplest possible implementation. For the duplicate threshold itself, a stricter threshold (≥ 0.99, near-exact match only) and a looser threshold (≥ 0.90) were both considered.

Options rejected. Brute-force pairwise comparison was rejected on complexity grounds: at $3{,}464$ prompts, this is on the order of 6 million comparisons, workable here but not something that scales to future corpus growth, whereas a FAISS index amortizes this cost. On the threshold, 0.99 was rejected because manual inspection showed many single-word paraphrases (e.g., "Develop" vs. "Outline" a plan) scoring around 0.95–0.98 while being functionally identical prompts; a 0.99 threshold would have missed most of these. A threshold of 0.90 was rejected in the other direction: at that looser threshold, manual spot-checks showed an increasing rate of prompts that were topically related but not truly duplicative (e.g., two different phishing-email requests), which would have overstated the duplication problem.

Decision taken. A FAISS flat inner-product index over L2-normalized embeddings (equivalent to cosine similarity), with a similarity threshold of 0.95, chosen as the point where manual review confirmed nearly all flagged pairs were genuine near-duplicates rather than merely topically similar prompts. This identified 546 duplicate pairs, of which 32 were cross-source (Section 4.3).

3.5 Decision: Clustering Algorithm and Dimensionality Reduction

Alternatives considered. k-means was the most obvious alternative to HDBSCAN, along with agglomerative hierarchical clustering. For dimensionality reduction ahead of clustering, PCA was the standard alternative to UMAP.

Options rejected. k-means was rejected because it requires specifying the number of clusters k in advance — for a corpus of unknown behavioral structure, this would mean guessing k rather than discovering it, and k-means forces every point into some cluster even when it is genuinely noise (e.g., an idiosyncratic one-off prompt with no behavioral neighbors). Agglomerative clustering was considered but rejected primarily due to its $O(n^2)$ memory and time behavior, which becomes a bottleneck if the corpus grows through future dataset additions. PCA was rejected as the dimensionality-reduction step because it is a linear projection and does not preserve the non-linear neighborhood structure that density-based clustering relies on; early experimentation showed PCA-reduced embeddings produced visibly less separable cluster structure than UMAP-reduced embeddings at the same target dimensionality.

Decision taken. UMAP (5 components, `n_neighbors=15`, `min_dist=0.0`) for dimensionality reduction, followed by HDBSCAN (Euclidean distance) for clustering. A separate 2-dimensional UMAP projection was computed purely for visualization; clustering itself was never performed in 2D, since 2D is not expressive enough to preserve the neighborhood structure that determines behavioral similarity — conflating the visualization space with the clustering space would have meant judging cluster quality from a deliberately lossy view of the data.

Within HDBSCAN, the `min_cluster_size` parameter required its own decision, resolved via a parameter sweep over $\{5, 10, 15, 25, 50\}$ (Table 2). `min_cluster_size` $= 10$ produced a marginally higher silhouette score (0.664 vs. 0.649) but was rejected because it yielded 99 clusters — judged too many to label and interpret meaningfully within the project timeline. `min_cluster_size` $= 50$ was rejected in the other direction: at only 16 clusters, manual review showed it merging behaviorally distinct attack types (e.g., DAN-style roleplay jailbreaks and financial-fraud prompts) into single clusters, which would have destroyed exactly the behavioral granularity the registry stage depends on. `min_cluster_size` $= 15$ was chosen as the point where each resulting cluster could be described in one interpretable sentence by a human reviewer — interpretability, not silhouette score, was treated as the deciding criterion, since silhouette score has no way of knowing whether a cluster boundary corresponds to a meaningful behavioral distinction.

Table 2: HDBSCAN parameter sweep results (`min_samples`=5, fixed).

min_cluster_size	# Clusters	Noise Rate (%)	Silhouette
5	150	21.3	0.663
10	99	23.4	0.664
15	**70**	**24.0**	**0.649**
25	43	26.0	0.619
50	16	11.9	0.504

A secondary parameter, `cluster_selection_epsilon`, was also swept (over $\{0.00, 0.05, 0.10, 0.20, 0.30, 0.50\}$) to test whether merging nearby micro-clusters improved interpretability. Higher epsilon values were rejected after

review showed them merging clusters that a human reader would consider behaviorally distinct, purely because they were geometrically adjacent in embedding space; `epsilon` $= 0.00$ (no forced merging) was retained as the operating default.

Cluster membership confidence was validated after the fact using HDBSCAN's soft-membership probabilities (points below 0.05 probability flagged as weak members), rather than trusting hard cluster labels at face value. This choice — treating HDBSCAN's own output as something to audit rather than accept — is what surfaced the mislabeled Cluster 7 (Section 4.2); a pipeline that only inspected hard labels would not have caught it.

3.6 Decision: Evaluating Cluster Quality Without Ground Truth

Alternatives considered. The most standard way to validate a clustering result is a supervised metric such as F1 or Adjusted Rand Index against a ground-truth partition. The `attack_category` field present in three of the five datasets was the only candidate for such a ground truth.

Option rejected. Using `attack_category` as ground truth to score the HDBSCAN clusters was considered and rejected as circular: cluster interpretation and labeling was informed by reading representative prompts from each cluster, and those prompts' original categories were often visible during that review — meaning the "ground truth" and the thing being evaluated were not independent. This is compounded by the fact that `attack_category` is entirely absent for two of the five datasets (AdvBench, In-The-Wild) and is not a unified taxonomy even where present, so it could not have scored the full corpus regardless.

Decision taken. Cluster quality was instead assessed through two independent, non-circular checks: interpretability review (can a human reviewer summarize the cluster in one sentence) at the tuning stage, and soft-membership-probability inspection (Section 3.5) to flag likely mislabeled or boundary-straddling clusters after assignment. Neither produces a single headline accuracy number, which is itself a limitation discussed in Section 5, but both are free of the circularity that using `attack_category` directly would have introduced.

3.7 Decision: One Taxonomy or Two

Alternatives considered. The simplest registry design would assign each cluster a single label describing the attack (e.g., "roleplay jailbreak for fraud"), collapsing technique and objective into one flat taxonomy.

Option rejected. A single flat taxonomy was rejected because it cannot represent the many-to-many relationship between technique and objective: roleplay framing (a technique) is used toward sexual-content generation, content-policy circumvention, and cybercrime objectives alike in this corpus; collapsing these into single combined labels would have produced a combinatorial explosion of near-duplicate categories ("roleplay-for-fraud", "roleplay-for-cybercrime", etc.) and made it impossible to ask which technique–objective combinations are *missing*, since "missing" is only a well-defined question against a fixed 2D grid.

Decision taken. Two orthogonal taxonomies — **primitive** (*how* the attack is constructed) and **behavior** (*what* outcome is pursued) — with each of the 69 clusters manually assigned exactly one of each, recorded in `cluster_assignments.ya` as the single source of truth. This is what makes the central coverage-matrix analysis (Section 3.7) possible at all.

3.8 Coverage Matrix Construction

A primitive $\times$ behavior coverage matrix was built by counting, for each (primitive, behavior) pair, the number of clusters assigned to that combination. The alternative of leaving unobserved combinations as absent (rather than explicit zero) cells was rejected, since an absent cell is ambiguous between "no data exists for this combination" and "this combination was not yet computed"; explicit zeros make coverage gaps directly and unambiguously visible.

3.9 Summary of Key Decisions

Table 3 consolidates the decisions above for quick reference.

4 Findings

4.1 The Coverage Matrix Is Mostly Empty

Of all possible primitive–behavior combinations in the registry, 75% contain zero clusters. This is the central empirical result of the project. It indicates that, despite drawing on five independently constructed datasets, the combined corpus is far from a comprehensive sampling of the attack space: most technique–objective combinations that are conceptually possible (e.g., `emotional_engagement` paired with `self_harm_enablement`, a documented real-world pattern) have no representative examples in any of the five source datasets. The densest

Table 3: Key design decisions, alternatives rejected, and deciding factor.

Decision	Rejected alternative(s)	Deciding factor
First dataset ingested	In-The-Wild; HarmBench/Do-Not-Answer	Schema generality; JBB's derivation from AdvBench/HarmBench
Schema strategy	Dataset-specific code; common-field-only schema	Information loss vs. downstream code complexity
Record ID	Auto-increment; random UUID	Determinism + cross-source duplicate visibility
Error handling	Single outer try/except	Isolating failure to one row, not one dataset
Embedding model	`mpnet-base-v2`; hosted API	Iteration speed; data-sensitivity risk
Dedup search	Brute-force pairwise	Scalability via FAISS indexing
Dedup threshold	≥ 0.99; ≥ 0.90	Manual review of false negatives/positives at each threshold
Clustering algorithm	k-means; agglomerative	Unknown cluster count; noise handling; scalability
Dim. reduction	PCA	Non-linear neighborhood preservation
`min_cluster_size`	10; 50	Interpretability over raw silhouette score
Cluster validation	F1 vs. `attack_category`	Circularity + missing labels in 2/5 datasets
Taxonomy structure	Single flat taxonomy	Many-to-many technique/objective relationship

cells cluster heavily around `instruction_override` as a primitive, meaning that a small number of mechanically similar attack strategies account for a disproportionate share of the corpus's apparent diversity.

4.2 Cluster Quality: An Overly Narrow Label

Manual review of soft cluster-membership probabilities identified one mislabeled cluster. Cluster 7, initially labeled "Roleplay person jailbreak," was found to be too narrow: it also contained a distinct sub-pattern of fictional AI-persona jailbreaks (e.g., `DAN`-style and `AIM`-style prompts) that do not center on historical-figure roleplay at all. Notably, all nine of the weakest-confidence members of this cluster originated from the In-The-Wild dataset, consistent with that dataset's known lack of academic curation and greater stylistic heterogeneity relative to JailbreakBench or HarmBench.

4.3 Cross-Source Duplication Between Academic Benchmarks

Deduplication at a similarity threshold of 0.95 identified 546 near-duplicate prompt pairs across the corpus, of which 32 were cross-source duplicates — the same prompt appearing verbatim (similarity score 1.000 in several cases) in both AdvBench and JailbreakBench. Examples include near-identical instructions for bomb-making, network intrusion, and identity theft tutorials appearing under both dataset labels. This confirms, empirically, what is only implicitly acknowledged in JailbreakBench's own documentation: that it was partially derived from AdvBench. The remaining 514 same-source duplicate pairs, concentrated in AdvBench, differ by only a single word (e.g., "Develop" vs. "Outline"), suggesting they originate from templated or LLM-generated prompt variation rather than independent human authorship.

This finding matters beyond bookkeeping: any evaluation that treats AdvBench and JailbreakBench as independent test sets risks inflated apparent robustness, since a defense that has "seen" a prompt via one benchmark may trivially succeed against its near-identical twin in the other.

4.4 Novelty Scores Are Dominated by Encoding Artifacts

A novelty scoring pass (identifying prompts furthest from any existing cluster centroid) found that the highest-scoring, ostensibly "most novel" prompts in the corpus were disproportionately prompts using unusual encodings, character substitutions, or formatting artifacts rather than genuinely novel attack strategies. This is an important caution for anyone using embedding-distance-based novelty metrics on this type of corpus: distance from existing clusters correlates as much with surface-level textual noise as with substantive novelty of attack technique or objective.

5 Limitations

No ground-truth clustering evaluation. Standard supervised clustering metrics (e.g., F1 against a ground-truth partition) could not be applied to validate the HDBSCAN clusters directly. The only candidate ground-truth field, `attack_category`, originates from the datasets' own pre-clustering metadata; using it to score post-hoc

clusters would be circular in cases where cluster boundaries were themselves informed by that metadata, and is further compromised by the fact that `attack_category` is entirely absent for two of the five source datasets (AdvBench and In-The-Wild) and is not a unified taxonomy across the datasets that do provide it. Cluster quality was therefore assessed through interpretability review and soft-membership-probability inspection rather than a single quantitative accuracy score.

Inherited harmfulness labels are not independently verified. The `is_harmful` field in the normalized schema is populated only for JailbreakBench, taken directly from that dataset's own `Behavior` annotation (`harmful` vs. `benign`); the remaining four datasets have no equivalent label and default to `None`. This label was never independently re-validated during this project — it was ingested and passed through as-is. Manual review during cluster labeling surfaced several borderline cases where a prompt's source-dataset harmfulness classification did not clearly match its actual content: e.g., prompts phrased adversarially but requesting genuinely benign information, and prompts whose harm depends heavily on context the label strips away (a question that is harmless in an educational framing but was collected as part of a "harmful behaviors" set because of its topic rather than its actual request). Because harmfulness was not the variable under study — primitive and behavior were assigned independently of the inherited `is_harmful` field — this does not directly bias the coverage matrix. It does mean that any downstream use of this corpus that filters or reports on "harmful prompt" counts using the `is_harmful` field should treat those counts as inherited from source-dataset judgment calls rather than as independently audited ground truth, and that the true rate of mislabeled harmfulness across the full corpus was not systematically measured.

Manual primitive/behavior assignment. Each of the 69 clusters was assigned its primitive and behavior label by manual review rather than automated classification. This introduces subjectivity and does not scale automatically to new clusters discovered from future data.

Severity is unpopulated. None of the five source datasets provide severity ratings, and this field remains unpopulated across the entire corpus. The behavior $\times$ severity coverage dimension proposed during registry design (Appendix, ADR 005) could not be computed as a result.

Dataset scope. The corpus, at $3{,}464$ prompts across five datasets, reflects only a fraction of the total adversarial-prompt landscape currently in circulation, and In-The-Wild in particular was ingested from only a single dated snapshot of a continuously growing collection.

Coverage is measured in clusters, not prompts. The coverage matrix (Section 3.7) increments each cell by the number of *clusters* assigned to a given primitive–behavior pair, not by the number of prompts within those clusters. A cell containing one large, 400-prompt cluster is therefore indistinguishable in the matrix from a cell containing one small, 15-prompt cluster, even though the former represents substantially more evidence for that technique–objective combination. The "dense" vs. "sparse" distinction defined in ADR 005 (3+ clusters vs. 0–1) inherits this same limitation: a technique–objective pair could be flagged dense on cluster count while still being thin in absolute prompt volume, or vice versa.

Noise points are excluded from the registry entirely. At a 24.0% noise rate, approximately 830 of the $3{,}464$ aligned prompts were labeled -1 by HDBSCAN and never assigned to any cluster, meaning they received no primitive or behavior label and were excluded from both `cluster_assignments.yaml` and the coverage matrix. The headline finding that 75% of the primitive–behavior matrix is empty is therefore computed over only the clustered $\sim$76% of the corpus; it is possible that some of the technique–objective combinations appearing as zero-coverage gaps are in fact represented among the unclustered noise points, just not densely enough for HDBSCAN to have recognized them as a coherent group.

English-only, text-only scope. All five source datasets consist exclusively of English-language, plain-text prompts. Neither non-English adversarial prompts nor multimodal attacks (e.g., image-embedded jailbreaks, audio-based prompt injection) are represented anywhere in the corpus, the embedding model, or the resulting taxonomy. The primitive and behavior categories developed here should therefore be understood as characterizing the English-language, text-only slice of the adversarial-prompt landscape, not the full space of LLM attack surfaces currently documented in the literature.

6 Future Work

1. **Incorporate WildJailbreak and ToxicChat.** These two datasets, discussed but not ingested in the current corpus (Section 2), are the most natural next additions: WildJailbreak's scale and benign-prompt variants would help populate currently-empty coverage cells and extend over-refusal analysis, while ToxicChat's naturalistic, deployed-chatbot origin would add a source of real production adversarial behavior distinct from both the academic benchmarks and In-The-Wild's forum-sourced prompts.

2. **Targeted data augmentation.** Use the coverage matrix directly to prioritize collection or synthetic gen-

eration of prompts filling the sparsest non-zero and zero-coverage primitive–behavior cells, rather than continuing to sample from datasets that reinforce existing dense regions.

3. **Automated primitive/behavior classification.** Train a lightweight classifier on the manually labeled cluster assignments to automatically tag new incoming prompts against the existing taxonomy, reducing the manual-review bottleneck as the corpus grows.

4. **Cross-benchmark deduplication as standard practice.** Extend the deduplication methodology to additional benchmarks as they are released, and propose it as a standard pre-processing step before benchmarks are combined for evaluation, given the magnitude of undocumented overlap found between AdvBench and JailbreakBench.

5. **Severity labeling.** Introduce a manual or model-assisted severity rating pass to enable the behavior $\times$ severity coverage dimension that the current dataset cannot support.

6. **Encoding-robust novelty scoring.** Investigate novelty metrics that normalize for surface-level encoding artifacts before measuring embedding distance, to better isolate genuinely novel attack strategies from superficial text perturbations.

Acknowledgments

This work was completed as part of an eight-week undergraduate internship. The author thanks her supervisor for weekly review and guidance throughout the project.

References

[1] JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models. *JailbreakBench/JBB-Behaviors*, 2024.

[2] Zou, A., Wang, Z., Kolter, J. Z., & Fredrikson, M. Universal and Transferable Adversarial Attacks on Aligned Language Models. *AlignmentResearch/AdvBench*, 2023.

[3] Mazeika, M. et al. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. Center for AI Safety, 2024.

[4] Wang, Y. et al. Do-Not-Answer: A Dataset for Evaluating Safeguards in LLMs. *LibrAI/do-not-answer*, 2023.

[5] Shen, X. et al. "Do Anything Now": Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models. *TrustAIRLab/in-the-wild-jailbreak-prompts*, 2023.

[6] Jiang, L., Rao, K., Han, S., Ettinger, A., Brahman, F., Kumar, S., Mireshghallah, N., Lu, X., Sap, M., Choi, Y., & Dziri, N. WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models. *arXiv:2406.18510*, 2024.

[7] Röttger, P. et al. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. 2024.

[8] Lin, Z. et al. ToxicChat: Unveiling Hidden Challenges of Toxicity Detection in Real-World User-AI Conversations. 2023.

[9] Gehman, S., Gururangan, S., Sap, M., Choi, Y., & Smith, N. A. RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models. *EMNLP*, 2020.

[10] McInnes, L., Healy, J., & Astels, S. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11), 205, 2017.

[11] McInnes, L., Healy, J., & Melville, J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv:1802.03426*, 2018.

[12] Johnson, J., Douze, M., & Jégou, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.